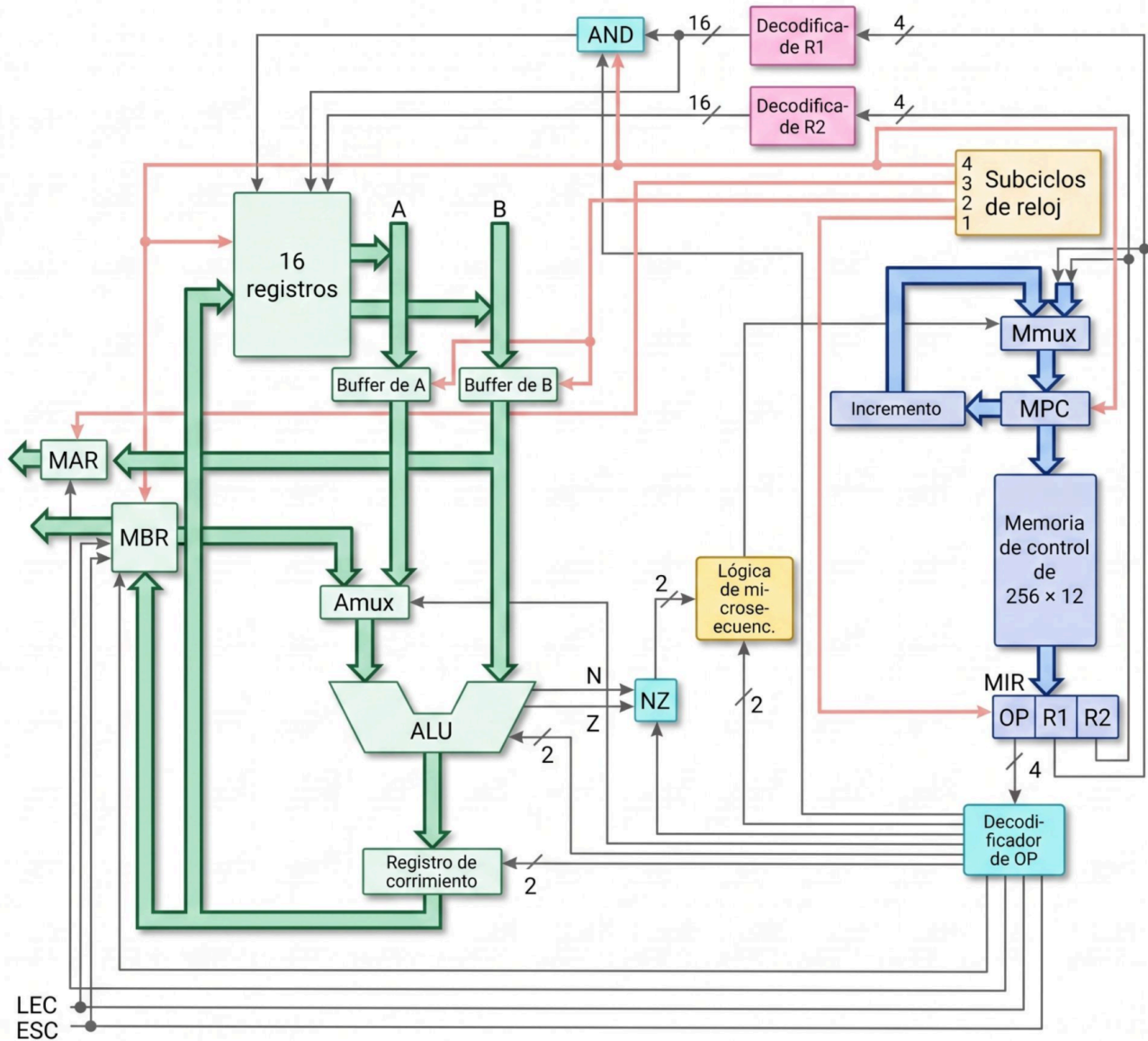


Fundamentos de Arquitectura de Computadoras

Arquitectura Vertical



Con registros:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PC	AC	SP	IR	TIR	0	+1	-1	AM ask	SM ask	Ax	Bx	Cx	Dx	Ex	Fx
					x00 00	x00 01	xFF FF	x0F FF	x00 FF						

AMUX

0 = Buffer A
1 = MBR

MSL (HL)

00 = No salta
01 = Si N==1
10 = Si Z==1
11 = Salta siempre

ALU (HL)

00 = A + B
01 = A & B
10 = A
11 = ~A

SH (HL)

00 = No desplaza
01 = alu >> 1
10 = alu << 1
11 = (no usado)

MBR, MAR, RD, WR, ENC

0 = No
1 = Si

Arquitectura Vertical - Microinstrucciones

Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	r1:=r1+r2
0001	AND	Y bit a bit	r1:=r1&r2
0010	MOVE	Mueve registro	r1:=r2
0011	COMPL	Complemento	r1:=inv(r2)
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1
0101	RSHIFT	Desplaza a la Derecha	r1:=r2>>1
0110	GETMBR	Almacena MBR en registro	r1:=MBR
0111	TEST	Examina registro	N:=(r2<0); Z:=(r2==0)
1000	BEGRD	Comienza la Lectura	MAR:=r1; rd
1001	BEGWR	Comienza la Escritura	MAR:=r1; MBR:=r2; wr
1010	CONRD	Continúa la Lectura	rd
1011	CONWR	Continúa la Escritura	wr
1100			
1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2
1110	ZJUMP	Salta si Z=true	if Z then goto r1<<4 r2
1111	UJUMP	Salta siempre	goto r1<<4 r2

Decodificador de operaciones

[illegible]

Fundamentos de Arquitectura de Computadoras - Arquitectura Vertical - Microinstrucciones

```
0: mar := pc; rd;
1: rd;
  pc := pc + 1;
2: ir := mbr;
  tir := lshift(tir);
  if n then goto 28;
3: tir := lshift(tir);
  if n then goto 19;
4: tir := lshift(tir);
  if n then goto 11;
5: alu := tir;
  if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
  goto 0;
9: mar := ir; mbr:= ac; wr;
10: wr; {STOD}
  goto 0;
11: alu := tir;
  if n then goto 15;
12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
  ac := ac + a;
  goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
  a := inv(a);
18: ac := ac + a;
  goto 0;
19: tir := lshift(tir);
  if n then goto 25;
20: alu := tir;
  if n then goto 23;
21: alu := ac; {JPOS}
  if n then goto 0;
22: pc := ir;
  pc band(pc, amask);
  goto 0;
23: alu := ac; {IZER}
  if z then goto 22;
24: goto 0;
25: alu := tir;
  if n then goto 27;
26: pc:= ir; {JUMP}
  pc:= band(pc, amask);
  goto 0;
27: ac := ir; {LOCO}
  ac :=band(ac, amask);
  goto 0;
28: tir := lshift (tir);
  if n then goto 40;
29: tir := lshift (tir);
  if n then goto 35;
30: alu := tir;
  if n then goto 33;
31: a := ir; {LODL}
  a := a + sp;
32: mar := a; rd;
  rd;
  ac := mbr;
  goto 0
33: a := ir; {STOL}
  a := a + sp;
34: mar := a; mbr := ac, wr;
  wr;
  goto 0;
35: alu := tir;
  if n then goto 38;
36: a := ir; {ADDL}
  a := a + sp;
37: mar := a; rd;
  rd;
  a := mbr;
  ac := ac + a;
  goto 0;
38: a := ir; {SUBL}
  a := a + sp;
39: mar := a; rd;
  rd;
  goto 99;
40: tir := lshift(tir);
  if n then goto 46;
41: alu:= tir;
  if n then goto 44;
42: alu:= ac; {JNEG}
  if n then goto 22;
43: goto 0;
44: alu := ac; {JNZE}
  if z then goto 0;
45: pc := ir;
  pc := band (pc, amask);
  goto 0;
46: tir := lshift(tir);
  if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
  wr;
49: pc := ir;
  pc := band(pc, amask);
  goto 0;
50: tir := lshift (tir );
  if n then goto 65;
51: tir := lshift(tir);
  if n then goto 59;
52: alu := tir;
  if n then goto 56;
53: mar := ac; rd; {PSHI}
  rd;
54: sp := sp + (-1);
55: a := mbr;
  mar := sp; mbr:=a, wr;
  wr;
  goto 0;
56: mar := sp; rd; {POPI}
57: rd;
  sp := sp + 1;
58: a := mbr;
  mar := ac; mbr := a; wr;
  wr;
  goto 0;
59: alu := tir;
  if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
  wr;
  goto 0;
62: mar := sp; rd; {POP}
63: rd;
  sp := sp + 1;
64: ac := mbr;
  goto 0;
65: tir := lshift (tir );
  if n then goto 73;
66: alu := tir;
  if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
  sp := sp + 1;
69: pc := mbr;
  goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp:= a;
  goto 0;
73: alu := tir;
  if n then goto 76;
74: a := ir; {INSP}
  a := band(a, smask);
75: sp := sp + a;
  goto 0;
76: a := ir; {DESP}
  a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
  sp := sp + a;
  goto 0;
```